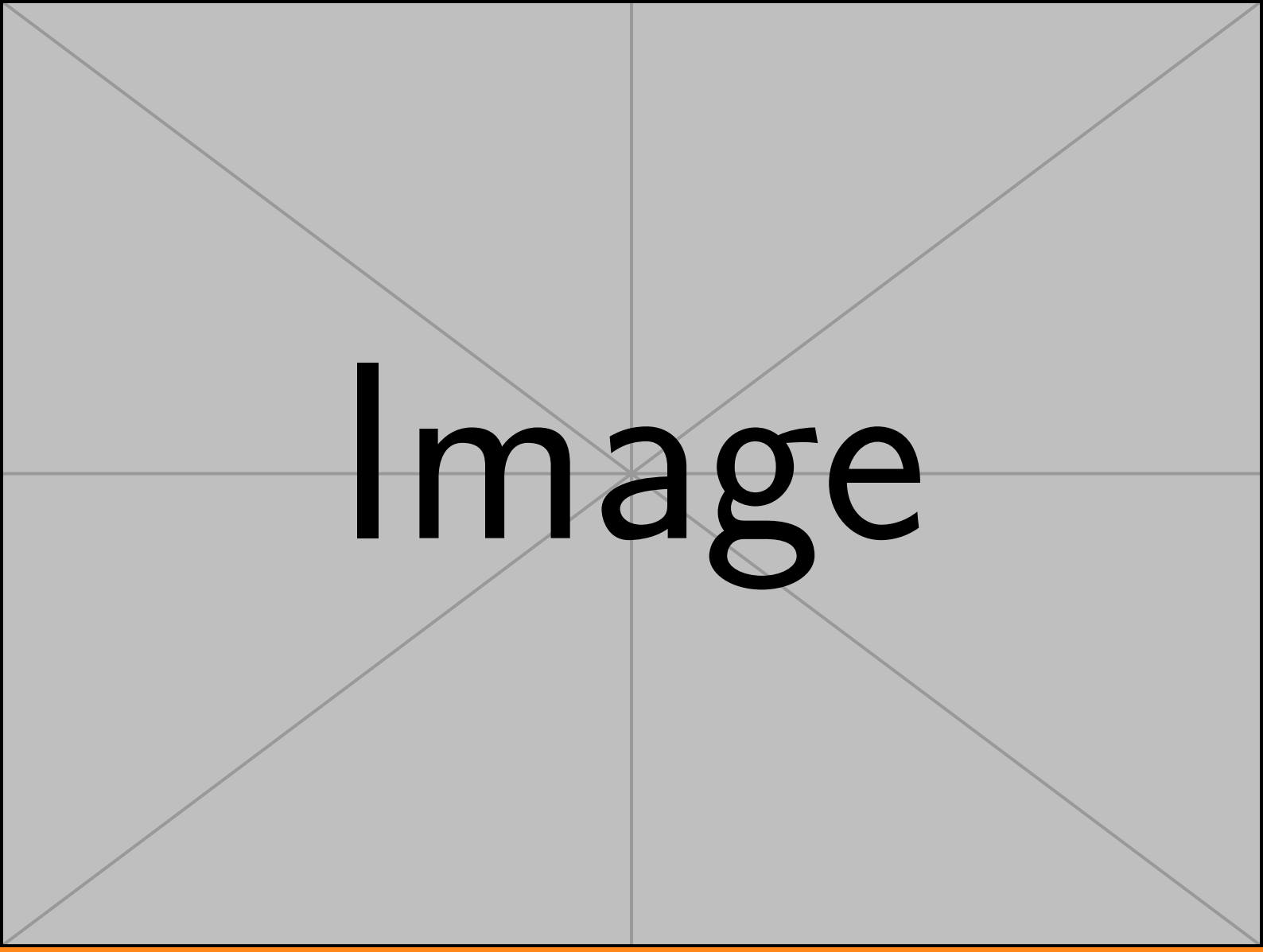


Modul Perkuliahan Desain Web

Panduan Praktik dan Latihan Terstruktur

Author: Tim Pengampu Mata Kuliah Desain Web

Institute: Program Studi / Fakultas

Date: October 13, 2026

Version: 1.0

Penggunaan: Untuk kalangan mahasiswa

Daftar Isi

Bab 1 Git & GitHub Dasar: Init, Commit, Push, Pull **1**

1.1 Apa itu Git? . . . 1
1.2 Apa itu GitHub? . . . 1
1.3 Alur Kerja Dasar . . . 1
1.4 Praktikum 1 — Memeriksa Instalasi Git . . . 2
1.5 Praktikum 2 — Konfigurasi Git . . . 2
1.6 Praktikum 3 — Inisialisasi Repository . . . 3
1.7 Praktikum 4 — Membuat File `.gitignore` . . . 4
1.8 Praktikum 5 — Commit Pertama . . . 4
1.9 Praktikum 6 — Membuat Repository di GitHub . . . 5
1.10 Praktikum 7 — Menghubungkan dan Push . . . 5
1.11 Praktikum 8 — Mengubah File dan Commit Lagi . . . 7
1.12 Praktikum 9 — Pull (Mengambil Perubahan) . . . 7
1.13 Latihan Mandiri . . . 8
1.14 Hasil yang Diharapkan . . . 8
1.15 Troubleshooting . . . 9
1.16 Rangkuman . . . 9

Daftar Gambar

1.1 Hasil `git --version` di terminal (menunjukkan versi Git terinstal). 2
1.2 Hasil `git config --list` (menampilkan `user.name` dan `user.email`). 3
1.3 Hasil `git status` setelah `git init` (file untracked). 3
1.4 Hasil `git status` setelah `git add .` (file *staged*, berwarna hijau). 5
1.5 Hasil `git log --oneline` (riwayat *commit*). 5
1.6 Halaman *repository* di GitHub setelah *push* pertama (berisi file proyek). 6
1.7 Halaman *repository* di GitHub setelah *push* pertama (berisi riwayat commit). 7
1.8 Hasil `git pull` yang berhasil mengambil perubahan dari GitHub. 8

Daftar Tabel

1.1 Masalah umum Git dan solusinya 9

Bab 1 Git & GitHub Dasar: Init, Commit, Push, Pull

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- menjelaskan kegunaan Git dan GitHub;
- menginstal dan mengonfigurasi Git di komputer;
- menginisialisasi *repository* Git (`git init`);
- melakukan *commit* dengan pesan yang baik;
- membuat *repository* di GitHub dan menghubungkannya;
- melakukan *push* dan *pull*.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- memeriksa instalasi Git dan mengonfigurasi identitas;
- menginisialisasi *repository* dan melakukan *commit* pertama;
- membuat *repository* di GitHub dan menghubungkannya dengan *repository* lokal;
- mempraktikkan alur kerja `git add` → `git commit` → `git push` → `git pull`.

1.1 Apa itu Git?

Git adalah **sistem kontrol versi** (*version control system*). Git melacak setiap perubahan pada file, sehingga Anda bisa:

- melihat riwayat perubahan;
- kembali ke versi sebelumnya;
- bekerja bersama orang lain tanpa menimpa pekerjaan.

1.2 Apa itu GitHub?

GitHub adalah **layanan hosting** untuk *repository* Git di internet. GitHub memungkinkan Anda menyimpan kode di cloud, berkolaborasi, dan (nanti) men-*deploy* website.

1.3 Alur Kerja Dasar

Listing 1.1: Alur kerja Git

```
Working Directory -> (git add) -> Staging Area
  -> (git commit) -> Repository Lokal
  -> (git push) -> GitHub
```

- **Working directory** — file yang sedang Anda edit.
- **Staging area** — file yang “dipilih” untuk di-*commit*.

- **Repository lokal** — riwayat *commit* di komputer Anda.
- **GitHub** — salinan *repository* di internet.

1.4 Praktikum 1 — Memeriksa Instalasi Git

Langkah Kerja

1. Buka terminal di VSCode (menu *Terminal* → *New Terminal*).
2. Ketik perintah berikut untuk memeriksa versi Git.

Listing 1.2: Memeriksa versi Git

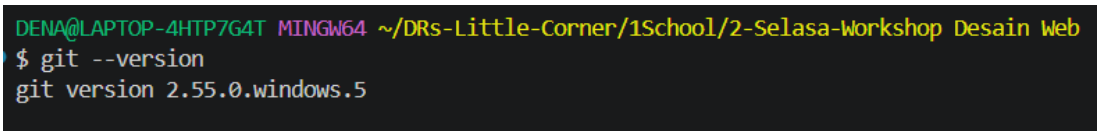
```
git --version
```

Langkah Kerja

1. Jika muncul versi (mis. `git version 2.40.0`), Git sudah terinstal. Jika belum, unduh dari `https://git-scm.com/downloads`, instal dengan pengaturan default, lalu ulangi Langkah 2.

Catatan

Saat instalasi, pilih editor default sesuai keinginan (mis. VSCode) dan biarkan opsi lain default.



Gambar 1.1: Hasil `git --version` di terminal (menunjukkan versi Git terinstal).
Jalankan `git --version` di terminal dan pastikan muncul nomor versi.

1.5 Praktikum 2 — Konfigurasi Git

Sebelum mulai, Git perlu tahu **siapa Anda**. Ini akan tercatat di setiap *commit*.

Langkah Kerja

1. Di terminal, ketik (ganti dengan nama dan email Anda):

Listing 1.3: Konfigurasi identitas Git

```
git config --global user.name "Nama␣Anda"
git config --global user.email "email@contoh.com"
```

Langkah Kerja

1. Gunakan email yang sama dengan akun GitHub Anda.
2. Periksa konfigurasi:

Listing 1.4: Memeriksa konfigurasi

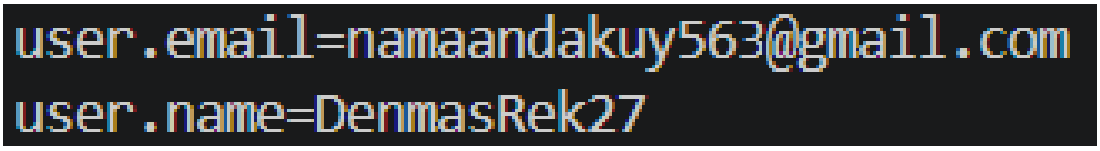
```
git config --list
```

Langkah Kerja

1. Pastikan `user.name` dan `user.email` muncul dengan benar.

Tips

Konfigurasi `--global` berlaku untuk semua proyek di komputer Anda. Lakukan sekali saja.



Gambar 1.2: Hasil `git config --list` (menampilkan `user.name` dan `user.email`).
Jalankan `git config --list` dan pastikan identitas Anda tercatat.

1.6 Praktikum 3 — Inisialisasi Repository

Langkah Kerja

1. Buka folder proyek yang sudah ada, misalnya `latihan-css` dari Bab 5.
2. Di terminal, pastikan Anda berada di folder proyek. Periksa dengan:

Listing 1.5: Memeriksa direktori kerja

```
pwd
```

Langkah Kerja

1. Inisialisasi *repository* Git:

Listing 1.6: Inisialisasi repository

```
git init
```

Langkah Kerja

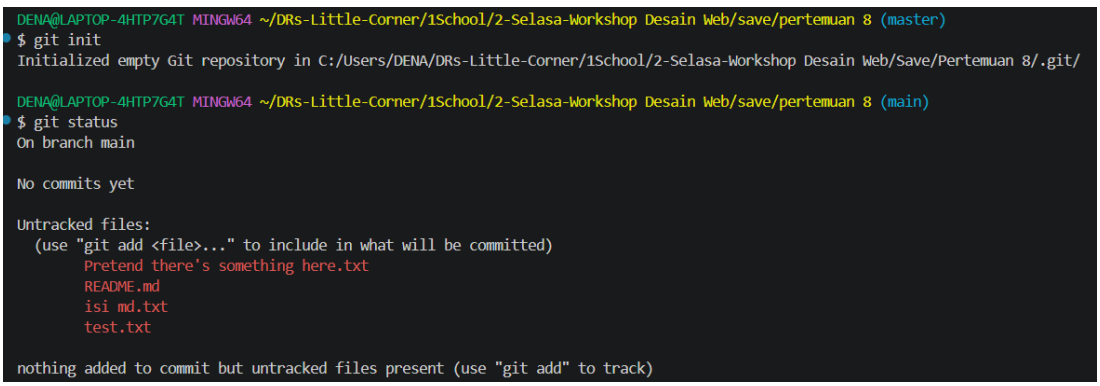
1. Periksa status *repository*:

Listing 1.7: Memeriksa status repository

```
git status
```

Catatan

`git init` membuat folder tersembunyi `.git` yang berisi riwayat versi. Jangan hapus folder ini.



Gambar 1.3: Hasil `git status` setelah `git init` (file untracked).
Jalankan `git status` setelah `git init` dan amati daftar file.

1.7 Praktikum 4 — Membuat File .gitignore

Langkah Kerja

1. Buat file bernama `.gitignore` di folder proyek.
2. Isi dengan file/folder yang **tidak perlu** di-*commit*:

Listing 1.8: Isi .gitignore

```
node_modules/
.DS_Store
*.log
```

Langkah Kerja

1. Simpan file.

Catatan

`.gitignore` memberi tahu Git file mana yang harus diabaikan. Berguna untuk file sementara atau sensitif.

1.8 Praktikum 5 — Commit Pertama

Langkah Kerja

1. Tambahkan semua file ke *staging area*:

Listing 1.9: Menambahkan file ke staging area

```
git add .
```

Langkah Kerja

1. Periksa status:

Listing 1.10: Memeriksa status setelah git add

```
git status
```

Langkah Kerja

1. File yang berwarna hijau (*staged*) siap di-*commit*. Lakukan *commit* pertama:

Listing 1.11: Commit pertama

```
git commit -m "Proyek␣awal␣latihan␣CSS"
```

Langkah Kerja

1. Periksa riwayat *commit*:

Listing 1.12: Melihat riwayat commit

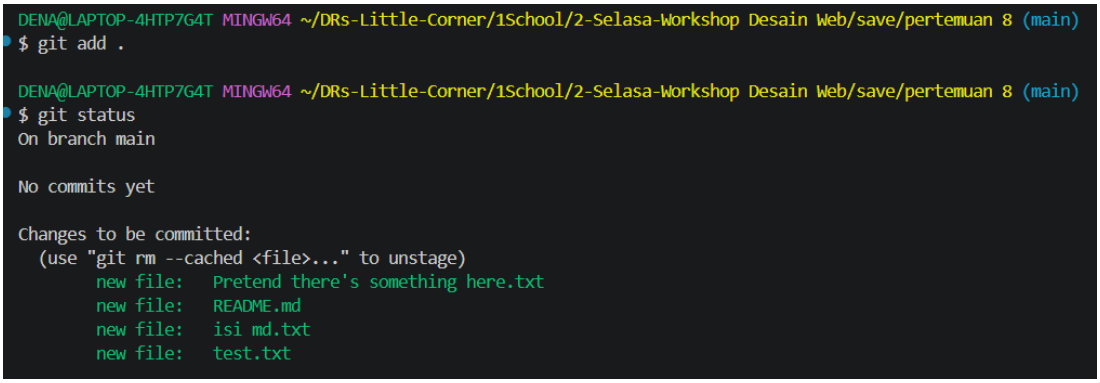
```
git log --oneline
```

Tips

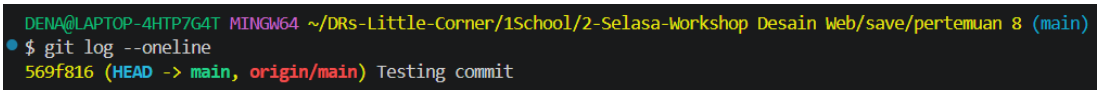
Tips penulisan pesan *commit* yang baik:

- Singkat dan jelas (maksimal ~50 karakter).

- Jelaskan **apa** yang diubah, bukan sekadar "update".
- Contoh: "Menambahkan halaman profil", "Memperbaiki bug navbar", "Mengubah warna tombol".



Gambar 1.4: Hasil `git status` setelah `git add .` (file *staged*, berwarna hijau).
Jalankan `git add .` lalu `git status` dan amati status file.



Gambar 1.5: Hasil `git log --oneline` (riwayat *commit*).
Jalankan `git log --oneline` dan pastikan *commit* pertama tercatat.

1.9 Praktikum 6 — Membuat Repository di GitHub

Langkah Kerja

1. Buka `https://github.com` dan *login*.
2. Klik tombol **+** di pojok kanan atas, lalu pilih **New repository**.
3. Isi:
   - **Repository name:** `latihan-css` (atau nama proyek Anda).
   - **Description:** (opsional) "Latihan CSS minggu 5".
   - **Visibility:** pilih **Public**.
   - **Jangan centang** "Add a README file" (karena kita sudah punya file).

Langkah Kerja

1. Klik **Create repository**.
2. Anda akan melihat halaman berisi perintah untuk menghubungkan *repository* lokal ke GitHub. Simpan halaman ini—kita akan menggunakannya di praktikum berikutnya.

Catatan

Jangan tutup halaman ini dulu. Perhatikan bagian "…or push an existing repository from the command line".

1.10 Praktikum 7 — Menghubungkan dan Push

Langkah Kerja

1. Kembali ke terminal VSCode.

2. Tambahkan *remote* (alamat *repository* GitHub). Ganti `USERNAME` dan `NAMA-REPO` sesuai milik Anda:

Listing 1.13: Menambahkan remote origin

```
git remote add origin https://github.com/USERNAME/NAMA-REPO.git
```

Langkah Kerja

1. Periksa *remote*:

Listing 1.14: Memeriksa remote

```
git remote -v
```

Langkah Kerja

1. *Push commit* ke GitHub. Ganti `main` dengan nama branch Anda jika berbeda:

Listing 1.15: Push ke GitHub

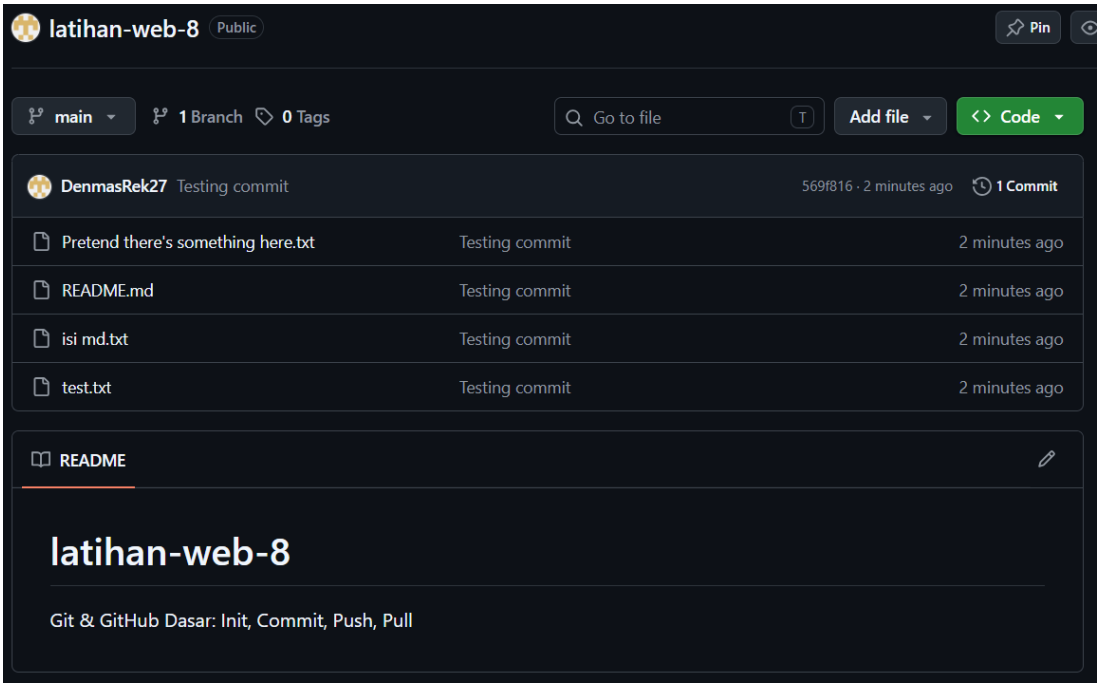
```
git branch -M main
git push -u origin main
```

Langkah Kerja

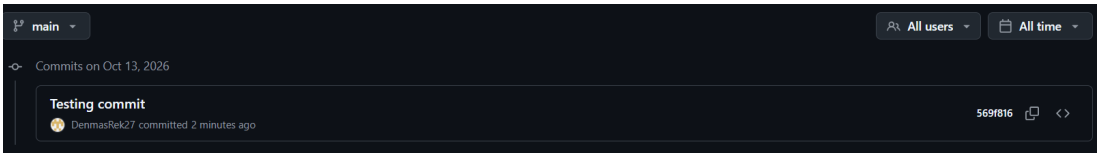
1. Jika diminta *login*, ikuti petunjuk (*browser* akan terbuka untuk autentikasi GitHub).
2. Buka halaman *repository* di GitHub dan *refresh*. File-file Anda sekarang ada di GitHub!

Catatan

`-u` (*upstream*) membuat Git mengingat bahwa branch `main` terhubung ke `origin/main`, sehingga *push* berikutnya cukup `git push`.



Gambar 1.6: Halaman *repository* di GitHub setelah *push* pertama (berisi file proyek). Buka halaman *repository* di GitHub dan pastikan file sudah muncul.



Gambar 1.7: Halaman *repository* di GitHub setelah *push* pertama (berisi riwayat commit).
Buka halaman *repository* di GitHub dan pastikan commit sudah muncul.

1.11 Praktikum 8 — Mengubah File dan Commit Lagi

Langkah Kerja

1. Ubah salah satu file di proyek Anda (mis. tambahkan paragraf baru di `index.html`).
2. Lihat perubahan yang terjadi:

Listing 1.16: Melihat perubahan

```
git status
git diff
```

Langkah Kerja

1. Tambahkan dan *commit* perubahan:

Listing 1.17: Commit perubahan baru

```
git add .
git commit -m "Menambahkan␣paragraf␣baru␣di␣index"
```

Langkah Kerja

1. *Push* ke GitHub:

Listing 1.18: Push ke GitHub

```
git push
```

Langkah Kerja

1. *Refresh* halaman GitHub dan lihat bahwa perubahan sudah masuk.

Catatan

Alur yang benar: `git add` → `git commit` → `git push`. Ulangi setiap kali selesai mengerjakan sesuatu.

1.12 Praktikum 9 — Pull (Mengambil Perubahan)

Langkah Kerja

1. Buka halaman *repository* di GitHub.
2. Klik file `index.html`, lalu klik ikon pensil (*edit*) di kanan atas.
3. Ubah sesuatu (mis. ganti judul), lalu klik **Commit changes** di bagian bawah.
4. Kembali ke terminal VSCode. Ambil perubahan dari GitHub:

Listing 1.19: Pull dari GitHub

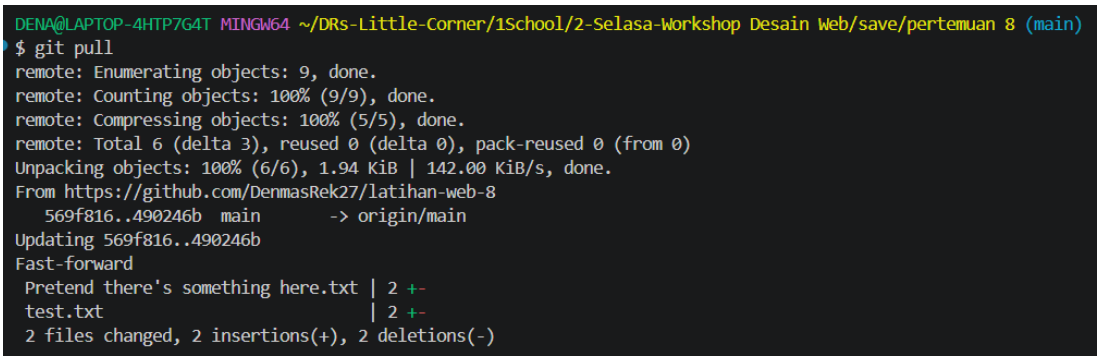
```
git pull
```

Langkah Kerja

1. Buka `index.html` di VSCode—perubahan yang Anda buat di GitHub sekarang ada di komputer lokal.

Catatan

`git pull` mengambil perubahan terbaru dari GitHub ke komputer Anda. Lakukan ini sebelum mulai bekerja jika bekerja bersama tim.



Gambar 1.8: Hasil `git pull` yang berhasil mengambil perubahan dari GitHub. Jalankan `git pull` dan pastikan perubahan dari GitHub sudah sinkron.

1.13 Latihan Mandiri

Latihan 1.1 Repository Flexbox

- Buat *repository* baru di GitHub bernama `latihan-flexbox`.
- Hubungkan folder `latihan-flexbox` (dari Bab 6) ke *repository* tersebut.
- Lakukan *commit* dengan pesan yang jelas untuk semua file.
- *Push* ke GitHub.
- Ubah satu file, *commit*, dan *push* lagi.
- Buat perubahan di GitHub (edit langsung), lalu `git pull` di komputer.
- Periksa `git log --oneline` dan pastikan semua *commit* tercatat.

1.14 Hasil yang Diharapkan

Setelah menyelesaikan semua praktikum, Anda memiliki:

- Git terinstal dan terkonfigurasi dengan nama dan email Anda.
- *Repository* lokal `latihan-css` dengan minimal 2 *commit*.
- *Repository* di GitHub bernama `latihan-css` berisi semua file proyek.
- Kemampuan untuk melakukan `git add`, `git commit`, `git push`, dan `git pull`.
- Riwayat *commit* yang bisa dilihat di `git log --oneline` dan di halaman GitHub.

1.15 Troubleshooting

Tabel 1.1: Masalah umum Git dan solusinya

Masalah	**Penyebab**	**Solusi**
`git` bukan perintah yang dikenali	Git belum terinstal / tidak di PATH	Instal Git dari git-scm.com, *restart* terminal/VSCode
`git push` meminta *password* terus	Autentikasi belum diatur	Gunakan *login browser* saat diminta, atau atur *Personal Access Token*
`fatal: remote origin already exists`	*Remote* sudah pernah ditambahkan	Gunakan `git remote set-url origin <URL>` untuk mengganti
`fatal: not a git repository`	Bukan di folder proyek	Pastikan berada di folder yang sudah `git init` (periksa dengan `pwd`)
Push ditolak (*rejected*)	*Remote* punya *commit* yang tidak ada di lokal	Jalankan `git pull` dulu, lalu `git push` lagi
File tidak muncul di GitHub	Belum di-*commit* / di-*push*	Jalankan `git add .`, `git commit -m "..."`, lalu `git push`
Commit tidak tercatat	Belum `git add`	Jalankan `git add .` sebelum `git commit`
Branch bernama `master` bukan `main`	Versi Git lama	Jalankan `git branch -M main` sebelum *push*

1.16 Rangkuman

- Git = kontrol versi lokal; GitHub = hosting *repository* di internet.
- `git init` — mulai *repository*.
- `git add .` — pindahkan file ke *staging area*.
- `git commit -m "pesan"` — simpan *snapshot* perubahan.
- `git push` — kirim *commit* ke GitHub.
- `git pull` — ambil perubahan dari GitHub.
- `git status` — lihat kondisi *repository*.
- `git log --oneline` — lihat riwayat *commit*.
- Tulis pesan *commit* yang jelas dan singkat.